

PAYHSM

Soft HSM pour Paiements Securises

Analyse de Conception et Diagrammes UML

Projet de Fin d'Etudes (PFE) — Architecture Securisee

Auteur : Yassine Nourdine

nourddineyassine2002@gmail.com

Mai 2026

<https://github.com/Schaib03/payhsm>

Ubuntu 24.04 | OpenSSL 3.x | GCC | SoftHSMv2

1. Introduction et Contexte

PayHSM est un projet PFE concevant un module de securite materiel logiciel (Soft HSM) dedie aux paiements electroniques. Il s'appuie sur SoftHSMv2 comme socle PKCS#11 et y greffe trois couches originales : defense systeme, gestionnaire de clefs fragmentees, et primitives cryptographiques conformes ISO 9564, ISO 9797-1 et EMV Book 2.

Problematiche : garantir la confidentialite de la LMK et des operations de paiement (PIN, MAC, EMV) sans puce HSM physique, en environnement Linux.

1.1 Perimetre

Analyse exclusive des modules PayHSM originaux (src/lib/payhsm/). SoftHSMv2 est utilise comme fondation PKCS#11 mais n'est pas analyse ici. L'integration se fait via SecureDataManager.cpp et SoftHSM.cpp (modifies).

1.2 Structure des Sources

Repertoire	Fichiers cles	Role
defense/	defense.c, seccomp_policy.c	Anti-dump, anti-pttrace, secure_zero, filtre BPF syscalls
keymanager/	xor_fragment.c, lmk_store.c, kek_provider.c, key_vault.c, integrity.c, mutation.c	Fragmentation LMK P1/P2/P3, stockage AES-256-GCM, coffre clefs
payment/	pin.c, mac.c, emv.c, key_exchange.c, pkcs11_payment.cpp	PIN Block ISO 9564, MAC ISO 9797-1, EMV Book 2, ABI C PKCS#11
payhsm_core.c/h	payhsm_core.c, payhsm.h	Coeur HSM : provision, startup, operations bancaires, gestion cartes
payhsm_switch.c/h	payhsm_switch.c/h	Interface Switch : clefs fournies par le reseau (GCM 88 hex / ECB 32 hex)
bin/	payhsm-cli.c, payhsmmd.c, payhsm-httpd.c	CLI, daemon Unix socket, serveur HTTP REST
tests/	test_scenarios.c, test_emv_switch.c, test_complet.c	Tests d'integration : PIN, MAC, EMV, mutation, switch

2. Besoins Fonctionnels

2.1 Gestion du Cycle de Vie (FR1)

ID	Intitule	Description
FR1.1	Provisionnement HSM	Generer LMK 256 bits (RAND_bytes), derivier TMK/TPK/TAK par terminal, initialiser le coffre chiffre. Passphrase requise.
FR1.2	Provisionnement LMK seule	Genere uniquement la LMK. Les cles metier restent chez le Switch (payhsm_provision_lmk_only).
FR1.3	Demarrage securise	Charger LMK chiffree, KEK via PBKDF2-SHA256 (100k), AES-256-GCM, fragmenter P1/P2/P3, effacer LMK. Defenses armees en premier.
FR1.4	Arret propre	zero_all_fragments() + secure_zero() sur tous les buffers secrets avant terminaison du processus.
FR1.5	Identifiant de boot	boot_id incremental pour synchroniser l'etat HSM-Switch apres chaque redemarrage (payhsm_get_boot_id).

2.2 Gestion des Cles (FR2)

ID	Intitule	Description
FR2.1	Fragmentation LMK	LMK fragmentee : P1 (BSS) XOR P2 (heap) XOR P3 (DATA) = LMK. Invariant maintenu en permanence. LMK effacee apres fragmentation.
FR2.2	Persistence LMK (AES-256-GCM)	Fichier lmk.bin : sel(16) nonce(12) tag(16) CT(32). Tag garantit l'integrite. AES-ECB interdit pour la LMK.
FR2.3	Derivation KEK (PBKDF2)	KEK = PBKDF2-SHA256(passphrase, sel, 100 000 iterations). Resistance aux attaques dictionnaire.
FR2.4	Coffre (KeyVault)	32 entrees typees (LMK/TMK/TPK/TAK/ZMK/ZPK/PVK/IMK). Chaque cle chiffree sous LMK + KCV 3 octets.
FR2.5	Verification integrite	HMAC-SHA256 de reference calcule a la fragmentation. Verifie avant chaque recompose_for_op(). Echech = emergency_shutdown().
FR2.6	Mutation cyclique	XOR aleatoire P1/P3 + reallocation heap P2. Rend inutilisable tout snapshot memoire anterieur.
FR2.7	Calcul KCV	KCV = AES-ECB(key, 0x00...00)[0:3]. Verification visuelle de toute cle du coffre.

2.3 Traitement du PIN (FR3)

ID	Intitule	Description
FR3.1	Generation PIN Block	PIN Block ISO 9564 Fmt0 chiffre sous TPK (AES-128-ECB) depuis PIN + PAN. Simulation GAP ou demo.
FR3.2	Translation TPK -> ZPK	Dechiffre sous TPK, rechiffre sous ZPK. PIN jamais en clair hors buffers proteges.
FR3.3	Verification PIN (PVV VISA)	PVV = decimalise(AES(PVK, PAN+PSN+PIN))[0:4]. Retourne 0 si valide, -1 sinon.

ID	Intitule	Description
FR3.4	Verification PIN Block chiffre	Dechiffre PIN Block (TPK ou ZPK), calcule PVV et compare. PIN ne transite jamais en clair.
FR3.5	Translation ZPK -> ZPK	Translation inter-bancaire entre deux ZPK pour routage reseau (payhsm_translate_zpk_to_zpk).

2.4 Operations MAC (FR4)

ID	Intitule	Description
FR4.1	MAC Retail (TAK)	MAC ISO 9797-1 Algorithme 3 (Retail MAC) sous TAK pour l'integrite des messages terminaux.
FR4.2	MAC inter-bancaire (ZAK)	MAC sous ZAK pour messages de compensation inter-bancaires.
FR4.3	Verification MAC temps constant	Comparaison a temps constant (loop XOR, pas de court-circuit) — anti-timing attack.

2.5 Traitement EMV (FR5)

ID	Intitule	Description
FR5.1	Derivation MK-AC	MK-AC = HMAC-SHA1(IMK, PAN PSN)[0:16]. Methode A EMV Book 2 — cle unique par carte.
FR5.2	Derivation SK-AC	SK-AC = XOR(MK-AC, ATC 0x00...). Cle de session unique par transaction.
FR5.3	Calcul ARQC	ARQC = CMAC-AES(SK-AC, donnees_tx). Simule le cryptogramme de la puce EMV.
FR5.4	Verification ARQC	Recalcule ARQC cote HSM emetteur et compare au cryptogramme reçu.
FR5.5	Generation ARPC	ARPC = CMAC(MK-AC, ARQC XOR ARC). Reponse d'autorisation envoyee a la puce.
FR5.6	Verification ARPC (puce)	Verification de l'ARPC cote puce — authentication mutuelle HSM-carte.
FR5.7	Simulation achat EMV	Orchestre MK-AC -> donnees_tx -> SK-AC -> ARQC -> verify -> ARPC en un appel payhsm_emv_simulate_purchase().

2.6 Gestion des Cartes (FR6) et Interface Switch (FR7)

- FR6.1 — Enregistrement carte : PAN + PIN -> calcul PVV -> persistance dans cards.pvv
- FR6.2 — Emission PVV Core Banking : payhsm_corebanking_issue_pvv() calcule et retourne le PVV
- FR6.3 — Lecture PVV : payhsm_corebanking_get_pvv() pour verifier l'enrolement
- FR7.1 — Dechiffrement cles Switch : GCM blob 88 hex (IV+Tag+CT) ou ECB 32 hex
- FR7.2 — Derivation terminaux Switch : TPK et TAK depuis TMK fourni par le Switch
- FR7.3 — Toutes operations PIN/MAC/EMV disponibles sans coffre local (mode Switch)

2.7 Couche de Defense Active (FR8)

ID	Mecanisme	Detail d'implementation
FR8.1	Anti-dump (OS)	setrlimit(RLIMIT_CORE,{0,0}) + mlockall(MCL_CURRENT MCL_FUTURE) — pas de core dump, pas de swap des secrets sur disque.
FR8.2	Anti-pttrace (debug)	prctl(PR_SET_DUMPABLE,0) + prctl(PR_SET_NO_NEW_PRIVS,1) — attachement debugger bloque.
FR8.3	Detection debugger active	Lecture periodique /proc/self/status -> TracerPid. Si != 0 : emergency_shutdown().
FR8.4	Handlers signaux fatals	SIGSEGV/SIGABRT/SIGTERM/SIGINT/SIGBUS/SIGFPE/SIGILL -> zeroiser P3 + _exit(1) signal-safe.
FR8.5	Arret d'urgence	emergency_shutdown() : zero_all_fragments() + _exit(99) sans destructeurs C++.
FR8.6	Filtrage syscalls (seccomp BPF)	Whitelist BPF : seuls les syscalls necessaires sont autorises. Tout autre = SIGKILL.

2.8 Interface PKCS#11 — ABI C Exporte (FR9)

pkcs11_payment.cpp exporte depuis libsofthsm2.so (__attribute__((visibility("default")))).
Partage du processus : LMK fragmentee et defenses identiques pour tous les clients (Python ctypes, banking apps).

Fonction exportee	Role
PayHSM_is_ready()	Verifie que la fragmentation LMK est active (1=pret, 0=non)
PayHSM_PIN_translate(pin_block_in, pan, tpk, zpk, out)	Translation PIN Block TPK -> ZPK
PayHSM_ARQC_verify(sk_ac, tx_data, tx_len, arqc)	Verification ARQC sous SK-AC
PayHSM_MAC_calculate(msg, msg_len, tak, mac_out)	Retail MAC ISO 9797-1 Algo 3 sous TAK
PayHSM_EMV_derive_sk_ac(mk_ac, atc, sk_ac_out)	Derivation SK-AC depuis MK-AC + ATC
PayHSM_EMV_compute_arqc(sk_ac, tx_data, tx_len, arqc_out)	Calcul ARQC (simulation cote puce pour demo)
PayHSM_version()	Chaine de version de l'extension PayHSM

3. Besoins Non Fonctionnels

3.1 Securite

ID	Exigence	Critere de satisfaction
NFR1.1	LMK jamais en clair	LMK fragmentee P1/P2/P3 en permanence. En clair uniquement pendant la nano-fenetre de fragmentation. <code>secure_zero</code> avec barriere assembleur <code>asm</code> volatile.
NFR1.2	Chiffrement authentifie	AES-256-GCM obligatoire pour <code>lmk.bin</code> . AES-ECB interdit pour la LMK. Modification du fichier detectee par le tag GCM.
NFR1.3	KDF resistente	PBKDF2-SHA256, 100 000 iterations, sel aleatoire 128 bits. Attaques dictionnaire couteuses.
NFR1.4	Zeroisaton systematique	<code>secure_zero()</code> apres chaque usage de buffer secret (LMK, KEK, TPK, ZPK, PIN clair, MK-AC, SK-AC, IMK).
NFR1.5	Memoire verrouilee	<code>mlockall(MCL_CURRENT MCL_FUTURE)</code> : pages memoire non swappables sur disque.
NFR1.6	Pas de clef codees en dur	Toutes les valeurs crypto viennent de <code>RAND_bytes</code> ou de la passphrase. Regle de codage obligatoire.
NFR1.7	Surface d'attaque reduite	Filtre <code>seccomp BPF whitelist</code> : seuls les syscalls HSM necessaires autorises.
NFR1.8	MAC temps constant	<code>verify_mac()</code> : boucle XOR sans court-circuit — protection contre les attaques temporelles.

3.2 Performance

- NFR2.1 — OpenSSL 3.x avec AES-NI (acceleration materielle) pour AES, HMAC, CMAC, PBKDF2.
- NFR2.2 — PBKDF2 execute une seule fois au demarrage. Les operations temps reel (PIN, MAC, EMV) n'incluent pas de KDF couteuse.
- NFR2.3 — Recomposition LMK (`recompose_for_op`) : XOR 96 octets — microsecondes.
- NFR2.4 — Mutation cyclique (`mutation_apply`) : XOR + malloc — microsecondes.

3.3 Fiabilite

- NFR3.1 — Verification HMAC avant chaque recomposition — aucune operation avec fragments corrompus.
- NFR3.2 — Codes retour explicites : `PAYHSM_RC_OK=0`, `PAYHSM_RC_ERR=-1`, `PAYHSM_RC_NOT_INIT=-2`, `PAYHSM_RC_DECLINED=55`.
- NFR3.3 — Arret d'urgence immediat sur toute detection d'intrusion (debugger, fragments corrompus).

3.4 Portabilite et Interoperabilite

- NFR4.1 — Cible : Ubuntu 24.04 LTS, Linux 6.x. Mecanismes specifiques Linux (`prctl`, `mlockall`, `seccomp`, `/proc`).
- NFR4.2 — Compilateur GCC, flags `-Wall -Wextra -lssl -lcrypto -I src/lib/payhsm`.

- NFR5.1 — Conforme ISO 9564 Fmt0, ISO 9797-1 Algo 3, EMV Book 2 Methode A, PKCS#11.
- NFR5.2 — ABI C extern compatible Python ctypes et pkcs11-tool.

3.5 Maintenabilite

- NFR6.1 — Architecture 4 couches a responsabilites exclusives (C0=defense, C1=cles, C2=paieement, C3=dispatch).
- NFR6.2 — Conventions uniformes : prefixe payhsm_, codes retour int, structs typedefs, headers auto-contenus.
- NFR6.3 — Tests d'integration dans tests/ : scenarios PIN, EMV, mutation, switch.

4. Architecture du Systeme

4.1 Vue en Couches

Couche	Modules	Responsabilite
C4 — Applications	BankApp, GAP, Switch, CLI, HTTP	Clients externes : socket Unix, API C, PKCS#11, REST
C3 — Dispatch	payhsm_core, payhsm_switch, payhsmd	Orchestration flux, contexte HSM singleton, routage requetes
C2 — Paiement	pin.c, mac.c, emv.c, key_exchange.c	Primitives crypto monetiques : PIN Block, MAC, ARQC/ARPC, derivation cles
C1 — Cles	xor_fragment.c, lmk_store.c, kek_provider.c, key_vault.c, integrity.c, mutation.c	Protection et cycle de vie LMK, coffre cles derivees
C0 — Defense	defense.c, seccomp_policy.c	Protection systeme transversale — activee EN PREMIER avant tout secret

4.2 Ordre de Demarrage (Bootstrap)

L'ordre d'initialisation est strictement impose. Toute deviation constitue une vulnerability de securite.

- 1. anti_dump_setup() — desactive core dumps, mlocke memoire, installe handlers signaux
- 2. anti_ptrace_setup() — bloque ptrace et elevation de privileges
- 3. install_seccomp_filter() — filtre BPF whitelist syscalls
- 4. lmk_store_read_salt() — lit le sel depuis lmk.bin
- 5. kek_derive_from_passphrase() — PBKDF2-SHA256 (100k iterations)
- 6. lmk_store_load() — AES-256-GCM : dechiffre la LMK + verifie le tag
- 7. secure_zero(kek, 32) — KEK efface immediatement apres usage
- 8. integrity_set_reference(lmk) — HMAC-SHA256 de reference calcule
- 9. fragment_lmk(lmk) — $P1 \text{ XOR } P2 \text{ XOR } P3 = \text{LMK}$, dispersion tri-segment
- 10. secure_zero(lmk, 32) — LMK originale effacee de la memoire
- 11. vault_load() — coffre des cles chiffrees sous LMK

Note : A partir de l'etape 10, la LMK n'existe plus en clair. Seuls P1/P2/P3 disperses permettent de la reconstituer.

4.3 Pattern Operationnel (Recomposition a la Demande)

Toute operation cryptographique necessitant la LMK suit invariablement ce pattern protect-use-zero :

- check_integrity() — verification HMAC-SHA256 des 3 fragments avant recomposition
- recompose_for_op(lmk) — XOR temporaire P1/P2/P3 dans buffer local de pile
- [operation crypto sur buffer secret derive de lmk]

- `secure_zero(lmk, 32)` — effacement immediat apres usage, quelques microsecondes apres recomposition

5. Description Detaillee des Modules

5.1 defense.c — Couche de Defense Systeme

Fondation securitaire de PayHSM. Cinq mecanismes complementaires et orthogonaux, actives avant tout chargement de secret.

anti_dump_setup()

- `setrlimit(RLIMIT_CORE,{0,0})` : empeche la creation de core dumps contenant les fragments LMK
- `mlockall(MCL_CURRENT|MCL_FUTURE)` : toutes les pages memoire verrouillees en RAM (pas de swap)
- sigaction sur 7 signaux fatals — handler `zerolse P3 + _exit(1)` de maniere signal-safe

anti_ptrace_setup() et anti_ptrace_check()

- `prctl(PR_SET_DUMPABLE,0) + prctl(PR_SET_NO_NEW_PRIVS,1)` — ptrace bloque des le demarrage
- `anti_ptrace_check()` lit `/proc/self/status -> TracerPid`. Si `!= 0` : `emergency_shutdown()`

secure_zero(ptr, len)

- Pointeur volatile : le compilateur ne peut pas optimiser les ecritures
- Barriere assembleur asm volatile(" ::: \"memory\") garantit l'ecriture effective en memoire

emergency_shutdown(reason) et install_seccomp_filter()

- `emergency_shutdown` : `zero_all_fragments() + _exit(99)` — terminaison sans destructeurs
- seccomp BPF whitelist : tout appel systeme hors liste autorisee declenche SIGKILL du noyau

5.2 xor_fragment.c — Fragmentation LMK

La LMK (32 octets) est divisee en P1 (segment BSS, zero-init), P2 (heap, malloc, adresse variable), P3 (segment DATA, variable globale). L'invariant $P1 \text{ XOR } P2 \text{ XOR } P3 = \text{LMK}$ est maintenu. La dispersion dans 3 segments distincts contraint un attaquant effectuant un dump a analyser l'espace memoire entier et a retrouver les 3 parties. La fragmentation seule n'est pas un chiffrement mais augmente considerablement la complexite forensique.

5.3 integrity.c / mutation.c — Integrite et Anti-Forensique

`integrity.c` calcule HMAC-SHA256(lmk) comme reference lors de la fragmentation initiale. Avant chaque `recompose_for_op()`, re-calcule HMAC($P1 \text{ XOR } P2 \text{ XOR } P3$) et compare. Echec = `emergency_shutdown()` immediate.

`mutation.c` applique XOR aleatoire (`RAND_bytes`) sur P1 et P3, puis realloque P2 sur une nouvelle adresse heap. Le HMAC est recalculé et verifié après mutation. Les statistiques (`count`, `last_ts`) sont exposees au dashboard.

5.4 lmk_store.c / kek_provider.c — Persistance

Format `lmk.bin` : `sel(16o) | nonce_GCM(12o) | tag_GCM(16o) | ciphertext(32o)`. KEK = PBKDF2-SHA256(`passphrase`, `sel`, 100000). AES-256-GCM garantit confidentialite ET integrite (`tag`) — toute modification du fichier est detectee au dechiffrement.

5.5 key_vault.c — Coffre des Cles

payhsm_key_vault_t : tableau 32 entrees payhsm_key_entry_t. Chaque entree : id(48 chars), type(enum 8 valeurs), terminal_id(32 chars), enc[16] (chiffre sous LMK AES-ECB), kcv[3]. API : vault_find(type, terminal_id), vault_find_by_id(id), vault_save(), vault_load(). La LMK est recomposee brievement pour chiffrer/dechiffrer.

5.6 pin.c — ISO 9564 Format 0

PIN Block = (0x0N PIN_digits 0xF...) XOR (0x0000 PAN_12digits). Chiffrement AES-128-ECB sous TPK. Translation : dechiffre TPK -> extrait PIN -> reconstruit bloc clair -> chiffre ZPK. PIN jamais en clair hors buffers proteges. PVV VISA = decimalise(AES(PVK, PAN+PSN+PIN))[0:4].

5.7 mac.c — ISO 9797-1 Algorithme 3

Retail MAC : CBC-MAC sur tous les blocs du message, puis AES sur le dernier bloc sous TAK. Verification a temps constant (boucle XOR, pas de memcmp() court-circuit). Le calcul ZAK fonctionne de maniere identique avec la cle ZAK.

5.8 emv.c — EMV Book 2

Methode A : MK-AC = HMAC-SHA1(IMK, PAN||PSN)[0:16]. SK-AC = XOR(MK-AC, ATC padde 16o). ARQC = CMAC-AES(SK-AC, donnees_tx). ARPC = CMAC-AES(MK-AC, ARQC XOR ARC). payhsm_emv_simulate_purchase() orchestre tout en un appel, zeroise chaque secret des qu'il n'est plus utile.

5.9 key_exchange.c — Hierarchie des Cles

TPK = AES-CMAC(TMK, terminal_id||'TPK'). TAK = AES-CMAC(TMK, terminal_id||'TAK'). ZPK chiffre sous ZMK (AES-ECB) pour transport inter-bancaire. KCV = AES-ECB(key, 0x00...00)[0:3] pour verification visuelle.

5.10 pkcs11_payment.cpp — ABI C PKCS#11

Wrapper C++ avec extern "C" et __attribute__((visibility("default"))). Deux raisons d'un ABI C : (1) ajouter des CKM_VENDOR_ dans SoftHSMv2 necessite de modifier plusieurs tables — hors scope PFE ; (2) l'ABI C est trivial a appeler depuis Python via ctypes. Partage du processus : LMK fragmentee et defenses identiques pour tous les clients.

6. Diagrammes UML de Conception

Les codes PlantUML complets sont dans le fichier `payhsm_diagrammes.puml` (meme dossier). Rendez-les via <https://plantuml.com>, l'extension VS Code PlantUML, ou la CLI PlantUML.

6.1 Diagramme de Cas d'Utilisation (UseCase_PayHSM)

26 cas d'utilisation, 5 acteurs (Administrateur HSM, Application Bancaire, Terminal GAP, Switch Inter-Bancaire, Puce EMV), 6 packages fonctionnels. Relations des : UC01-Provision inclut UC05-Fragmentation, UC24-Anti-dump, UC25-Anti-pttrace. UC02-Startup inclut UC06-Integrite. UC21-Achat EMV inclut UC16, UC18, UC19.

Note : Section @startuml UseCase_PayHSM dans payhsm_diagrammes.puml

6.2 Diagramme de Classes (Classes_PayHSM)

20 classes/modules, 5 packages (Core, KeyManager, Payment, Defense, Switch Interface). Agregation forte : `payhsm_ctx_t` contient `payhsm_key_vault_t` (1:1), qui compose 0..32 `payhsm_key_entry_t`. `PKCS11Payment` delegue a `PinModule`, `MacModule`, `EmvModule`. `SwitchModule` reutilise les memes primitives que `Core`. `payhsm_key_type_t` : 8 valeurs (LMK, TMK, TPK, TAK, ZMK, ZPK, PVK, IMK).

Note : Section @startuml Classes_PayHSM dans payhsm_diagrammes.puml

6.3 Sequence — Demarrage du HSM (Sequence_Startup)

9 participants, 18 messages. Sequence : armement defenses (`anti_dump`, `anti_pttrace`, `seccomp`) -> lecture sel -> `PBKDF2-SHA256` -> `AES-256-GCM` dechiffrement -> `secure_zero(kek)` -> HMAC reference -> fragmentation `P1/P2/P3` -> `secure_zero(lmk)` -> `vault_load`. A la fin, la LMK n'existe plus en clair nulle part.

Note : Section @startuml Sequence_Startup dans payhsm_diagrammes.puml

6.4 Sequence — Verification PIN Flux GAP (Sequence_PIN_Verify)

Flux complet porteur -> terminal -> switch -> HSM acquéreur -> HSM emetteur. L'HSM acquéreur translate TPK -> ZPK. L'HSM emetteur verifie contre PVV. Chaque HSM commence par `check_integrity()` et termine par `secure_zero()` de tous les secrets manipules.

Note : Section @startuml Sequence_PIN_Verify dans payhsm_diagrammes.puml

6.5 Sequence — Paiement EMV (Sequence_EMV_Purchase)

Simulation complete en un appel `payhsm_emv_simulate_purchase()`. 10 etapes : recompose LMK -> derive MK-AC -> build donnees_tx -> derive SK-AC -> calcule ARQC -> verifie ARQC -> genere ARPC -> `secure_zero(lmk, mk_ac, sk_ac)`. Flux orchestre sans jamais exposer les cle en dehors des buffers locaux.

Note : Section @startuml Sequence_EMV_Purchase dans payhsm_diagrammes.puml

6.6 Activite — Provisionnement (Activity_Provision)

Flux lineaire de 12 etapes avec branchement conditionnel sur la presence de terminaux. Inclut : armement defenses, generation LMK aleatoire, derivation KEK `PBKDF2`, chiffrement `AES-256-GCM`, persistance `lmk.bin`, `secure_zero(kek)`, HMAC reference, fragmentation, `secure_zero(lmk)`, et si terminaux : derive TMK/TPK/TAK, KCV, `vault_add_key`, `vault_save`.

Note : Section @startuml Activity_Provision dans payhsm_diagrammes.puml

6.7 Activite — Translation PIN TPK->ZPK (Activity_PIN_Translation)

Deux gardes de securite : (1) HSM initialise ?, (2) Integrite HMAC OK ? -- echec = emergency_shutdown(). Si OK : recompose LMK -> dechiffre TPK et ZPK sous LMK -> secure_zero(lmk) -> AES-ECB decrypt sous TPK -> extraction ISO 9564 -> AES-ECB encrypt sous ZPK -> secure_zero(tpk, zpk, pin_plain).

Note : Section @startuml Activity_PIN_Translation dans payhsm_diagrammes.puml

6.8 Architecture Composants (Architecture_PayHSM)

5 couches, 3 stores persistants (lmk.bin, keys.vault, cards.pvv), 3 interfaces clientes (PKCS#11 C ABI, Unix Socket, REST HTTP), dependance OpenSSL 3.x transversale. La couche Defense est transversale : utilisee par Core via direct calls et par XorFragment pour zero_all_fragments().

Note : Section @startuml Architecture_PayHSM dans payhsm_diagrammes.puml

6.9 Sequence — Mutation des Fragments (Sequence_Mutation)

Flux : verify HMAC pre-mutation -> recompose verification -> secure_zero -> mutation_apply(XOR nonce + realloc heap P2 + MAJ HMAC) -> verify HMAC post-mutation -> lecture stats (count, last_ts). Ce flux garantit l'invariant d'integrite avant et apres chaque mutation.

Note : Section @startuml Sequence_Mutation dans payhsm_diagrammes.puml

7. Innovations, Conformite et Perspectives

7.1 Innovations Techniques

Innovation	Detail
Fragmentation LMK tri-partite	P1 (BSS) XOR P2 (heap, adresse variable) XOR P3 (DATA). Dispersion dans 3 segments force l'analyse de tout l'espace memoire du processus.
Mutation anti-forensique	XOR aleatoire periodique + reallocation heap. Tout snapshot memoire anterieur est inutilisable pour reconstruire la LMK.
Defense en profondeur (4 couches)	Anti-dump (OS) + anti-ptrace (debug) + seccomp BPF (syscalls) + HMAC fragments (crypto). Orthogonaux : un attaquant doit tous les contourner.
Zeroisation resistente compilateur	volatile + asm volatile barriere memoire. Contrairement a memset(), ne peut pas etre elimine par les optimisations du compilateur.
ABI C partagee PKCS#11	Partage du processus libsoftsm2.so : LMK fragmentee et defenses identiques pour tous les clients (Python ctypes, pkcs11-tool, banking apps).
Mode Switch complet	Toutes les operations PIN/MAC/EMV disponibles avec cles fournies par le Switch (GCM 88 hex / ECB 32 hex) sans coffre HSM local.

7.2 Conformite aux Standards

Standard	Implementation PayHSM
ISO 9564 Format 0 (PIN Block)	pin.c : generate_pin_block(), translate_pin_block(), verify_encrypted_pin_block()
ISO 9797-1 Algo 3 (Retail MAC)	mac.c : calculate_mac_tak(), calculate_mac_zak(), verify_mac()
EMV Book 2 Methode A	emv.c : derive_card_keys(), derive_sk_ac(), emv_compute_arqc(), generate_arpc(), verify_arpc()
PVV VISA	pin.c : pin_compute_pvv() — decimalisation AES/Triple-DES
PKCS#11 (via SoftHSMv2)	pkcs11_payment.cpp + integration SecureDataManager.cpp + SoftHSM.cpp
Linux Security (LSM/prctl/seccomp)	defense.c : prctl, mlockall, seccomp BPF, rlimit, /proc/self/status

7.3 Perspectives d'Evolution

- Authentification mutuelle TLS entre payhsm-cli et payhsmd (actuellement socket Unix local non chiffre).
- Integration TPM : stocker le KEK dans un TPM hardware pour un stockage de confiance materielle.
- Clustering multi-HSM : replication chiffee du coffre entre plusieurs instances.
- Certification PCI HSM : audit et certification formelle de l'implementation.
- Interface REST securisee : TLS mutuel + JWT pour payhsm-httpd.

8. Glossaire

Terme	Définition
ARPC	Authorization Response Cryptogram — réponse chiffrée de l'émetteur à la puce carte prouvant l'authenticité de la banque.
ARQC	Authorization Request Cryptogram — cryptogramme généré par la puce EMV, prouvant l'authenticité de la carte.
ATC	Application Transaction Counter — compteur de transaction unique par puce EMV (2 octets).
BPF	Berkeley Packet Filter — programme de filtrage noyau utilisé pour seccomp (filtrage syscalls).
CMAC	Cipher-based MAC sur AES — utilisé pour ARQC/ARPC EMV.
GAP	Guichet Automatique de Paiement — terminal équipé d'un EPP (Encrypting PIN Pad).
HSM	Hardware Security Module — module de sécurité dédié aux clés crypto. PayHSM en est une implémentation logicielle.
IMK	Issuer Master Key — clé maîtresse de la banque émettrice, racine des clés carte.
KCV	Key Check Value = AES-ECB(key, 0x00...)[0:3] — vérification visuelle d'une clé.
KEK	Key Encryption Key — dérivée de la passphrase (PBKDF2) pour chiffrer la LMK.
LMK	Local Master Key — clé maîtresse du HSM. Jamais en clair en mémoire (fragmentée P1/P2/P3).
MK-AC	Master Key Application Cryptogram — clé carte dérivée de l'IMK et du PAN/PSN.
PAN	Primary Account Number — numéro de carte bancaire (16 chiffres).
PBKDF2	Password-Based Key Derivation Function 2 — KDF robuste avec sel et itérations (ici SHA256, 100k).
PVK	PIN Verification Key — clé pour calculer et vérifier le PVV.
PVV	PIN Verification Value — 4 chiffres stockés dans le HSM émetteur pour vérifier le PIN sans le stocker.
PSN	PAN Sequence Number — numéro de séquence carte (distingue les reemissions).
SK-AC	Session Key Application Cryptogram = $f(\text{MK-AC}, \text{ATC})$ — unique par transaction EMV.
TAK	Terminal Authentication Key — clé de terminal pour calcul de MAC.
TMK	Terminal Master Key — dérivée TPK et TAK.
TPK	Terminal PIN Key — chiffrement du PIN Block au terminal.
ZAK	Zone Authentication Key — MAC inter-bancaire.
ZMK	Zone Master Key — échange de ZPK entre banques.
ZPK	Zone PIN Key — transport de PIN Blocks entre banques.

--- *Fin du Document* ---

PayHSM PFE 2026 | Analyse de Conception | Confidentiel